

COMMONWEALTH OF AUSTRALIA

Copyright Regulations 1969

This material has been reproduced and communicated to you by or on behalf of Monash University pursuant to Part VB of the Copyright Act 1968 (the Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice

Prepared by: [\[Arun Konagurthu\]](#)

Extended by: Taylor Kearney

FIT3155: Algorithms and Data Structures

Week 9: **B-Trees: A generalized balanced Search Tree**

Faculty of Information Technology, Monash University

What is covered in this lecture?

- A **generalization** of the balanced search trees
- **Standard operations** on B-trees (search, insert, delete)
- Space and time **complexity** issues

Source material and recommended reading

- Cormen et al. “Introduction to Algorithms” (Chapter 18) [\[link\]](#)

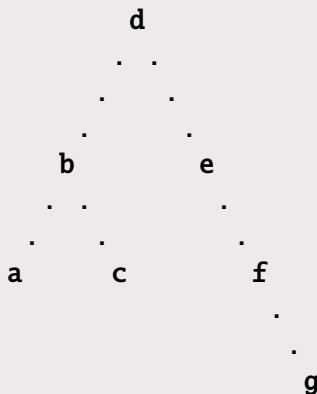
Original papers

- R. Bayer and E. McCreight
 - ▶ *Organization and Maintenance of Large Ordered Indices*, **Mathematical and Information Sciences** Report No. 20, Boeing Scientific Research Laboratories. 1970
- R. Bayer,
 - ▶ *Binary B-Trees for Virtual Memory*, **Proceedings of 1971 ACM-SIGFIDET Workshop on Data Description, Access and Control**, San Diego, California. November 11–12, 1971.

Revision from FIT2004: **Balanced Binary** Search Trees

- The empty tree is a balanced BST
- If the tree is not empty,
 - 1 the elements in the **left subtree** are $\leq$ the element in the root
 - 2 the elements in the **right subtree** are $>$ the element in the root
 - 3 the **left subtree is a balanced BST**
 - 4 the **right subtree is a balanced BST**

BST example



Detour: The memory hierarchy – non-examinable

CPU caches (**volatile**)

- Very fast (about 3-10 CPU clock cycles to access), but very expensive!

Primary storage/RAM (**volatile**)

- About $10\times$ - $100\times$ slower than CPU access to cache, but less expensive.

secondary storage (**non-volatile**):

- Traditional enterprise storage is based on magnetic storage technology (hard disk drives, HDDs).
 - ▶ Very cheap, but $10^5\times$ slower than access to primary storage.
- Lately, solid state drives (SSDs) are becoming increasingly popular.
 - ▶ Not that cheap; $10^3\times$ slower than access to primary storage.

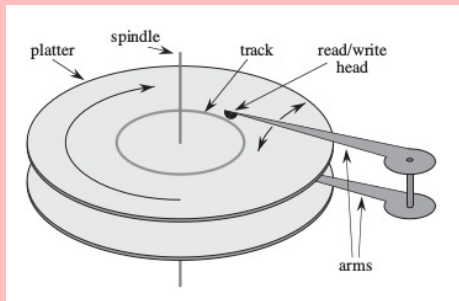
Detour: Memory latency along the hierarchy – non-examinable

Event	Latency (real)	Scaled (for intuition)
1 CPU cycle	0.3ns	1s
L1 cache access	0.9ns	3s
L2 cache access	2.8ns	9s
L3 cache access	12.9ns	43s
RAM access	100ns	5min
SSD I/O	20-100 μ s	1 - 4 days
HDD I/O	1-10ms	1-12 months

Detour: A typical hard disk drive – non-examinable

- A typical hard disk drive consists of:

- ▶ one or more **platters** covered with magnetizable material that rotate around a **spindle**.
- ▶ Read and write to each platter is via a **head** attached to an **arm**.
- ▶ The arms can move their heads towards or away from the spindle.
- ▶ When the arm is stationary, the surface below the head is called a **track**.



Detour: Read/Write Access times – non-examinable

- Secondary storage is much slower because they have moving parts
 - ▶ Platter rotations (5,400-15,000 RPM).
 - ▶ Arm head forward/backward movement.
- At 7,200 RPM, one rotation takes 8.33 milliseconds, which is...
- ... 5 orders of magnitude $>$ than 50 nanoseconds access time to main memory.
- That is, in time to wait for one read/write on a secondary storage, we can access primary memory 100,000 times during that span.
- Average access times for commodity disks are in the range 8 to 11 milliseconds.

Detour: Disk access a **page** worth of information at a time
– **non-examinable**

- To amortize the time wasted during mechanical movements, each disk access reads
 - ▶ not just one item but several items at a time.
- Information is divided (logically) into equal-sized **pages** of bits.
- Each access reads/writes one or more **pages** worth of information at a time.
- A page contains from 4KB to 64KB (in some cases) worth of information.
- The number of disk accesses is measured in terms of the number of pages of information.

Disk based search tree structures

- Consider the cases when we have large (dynamic) database/dictionary, too big to fit in main memory.
- Accessing its information, we have the following cost model to optimize:
 - ▶ Minimize expected number of disk accesses during various operations (insert/delete/search).
 - ▶ Keep space requirement to $O(n)$.
- Binary search trees are **not** optimal for disk-based representations.
- Generalization to multi-way search trees greatly reduce the number of node accesses.
- **B-tree** (or its variants) is a very practical, widely-used data structure for this purpose.

B-trees: Introduction

- B-trees **generalize** **balanced** search trees

B-trees: Introduction

- B-trees **generalize** **balanced** search trees
- By generalization, the tree is **no longer** **binary** but has **many branches per node**.

B-trees: Introduction

- B-trees **generalize** **balanced** search trees
- By generalization, the tree is **no longer binary** but has **many branches per node**.
- They are really powerful and are used on many mission critical systems that rely on a large amount of data stored on a **secondary storage** device (disk)

B-trees: Introduction

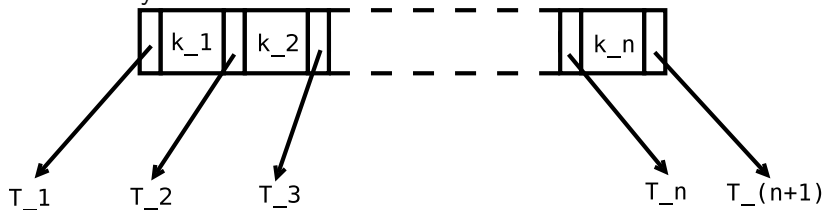
- B-trees **generalize** **balanced** search trees
- By generalization, the tree is **no longer binary** but has **many branches per node**.
- They are really powerful and are used on many mission critical systems that rely on a large amount of data stored on a **secondary storage** device (disk)
- Common examples are **very large databases** and **filesystems**

B-tree properties

- 1 A **B-tree** is a **rooted tree**.

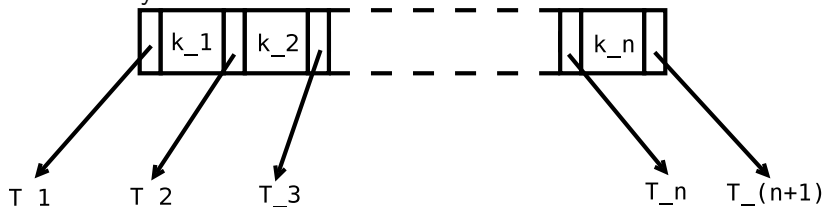
B-tree properties

- 1 A **B-tree** is a **rooted tree**.
- 2 An arbitrary node looks like this:



B-tree properties

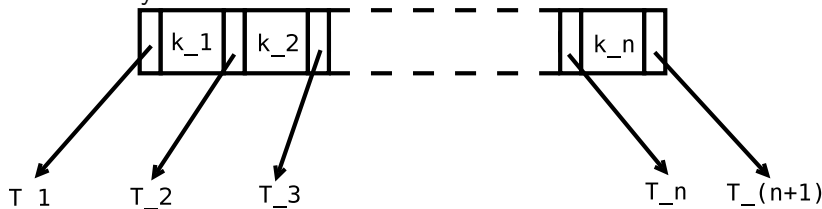
- 1 A **B-tree** is a **rooted tree**.
- 2 An arbitrary node looks like this:



- 3 It has n elements: $k_1 < k_2 < \dots < k_n$ that are stored in a **non-decreasing** (sorted) order.

B-tree properties

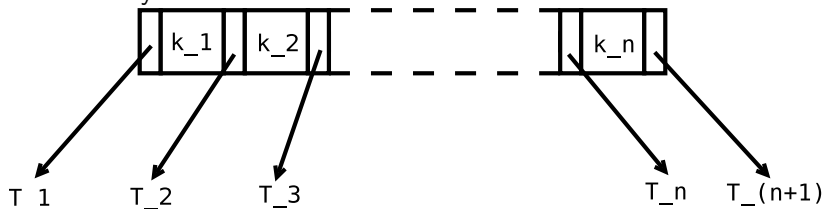
- 1 A **B-tree** is a **rooted tree**.
- 2 An arbitrary node looks like this:



- 3 It has n elements: $k_1 < k_2 < \dots < k_n$ that are stored in a **non-decreasing** (sorted) order.
- 4 It also stores $n + 1$ **pointers/links to subtrees**: $T_1, T_2, \dots, T_n, T_{n+1}$ that are **distributed regularly** between the elements.

B-tree properties

- 1 A **B-tree** is a **rooted tree**.
- 2 An arbitrary node looks like this:



- 3 It has n elements: $k_1 < k_2 < \dots < k_n$ that are stored in a **non-decreasing** (sorted) order.
- 4 It also stores $n + 1$ **pointers/links to subtrees**: $T_1, T_2, \dots, T_n, T_{n+1}$ that are **distributed regularly** between the elements.
- 5 Each successive (k_i, T_i, T_{i+1}) is analogous to a **fork**/node of a **binary** tree and has the BST **structural property**: $\{T_i\} < k_i < \{T_{i+1}\}$. So, generalizing this, we have:

$$\{T_1\} < k_1 < \{T_2\} < k_2 < \{T_3\} < \dots < \{T_n\} < k_n < \{T_{n+1}\}$$

B-tree properties...continued

- ⑥ All leaves have the **same depth** (from the root).

B-tree properties...continued

- ⑥ All leaves have the **same depth** (from the root).
- ⑦ **IMPORTANT!** The **number of elements per node** has a **lower bound** and an **upper bound**. These bounds are expressed using the parameter $t \geq 2$, also called the **minimum degree** of the B-tree:

B-tree properties...continued

- ⑥ All leaves have the **same depth** (from the root).
- ⑦ **IMPORTANT!** The **number of elements per node** has a **lower bound** and an **upper bound**. These bounds are expressed using the parameter $t \geq 2$, also called the **minimum degree** of the B-tree:

Lower bound: Each node must have **AT LEAST** t **subtrees** as children or equivalently $t - 1$ elements.

B-tree properties...continued

- ⑥ All leaves have the **same depth** (from the root).
- ⑦ **IMPORTANT!** The **number of elements per node** has a **lower bound** and an **upper bound**. These bounds are expressed using the parameter $t \geq 2$, also called the **minimum degree** of the B-tree:

Lower bound: Each node must have **AT LEAST** t **subtrees** as children or equivalently $t - 1$ elements.

- The only **exception** to this bound is for the **root** node.

B-tree properties...continued

- ⑥ All leaves have the **same depth** (from the root).
- ⑦ **IMPORTANT!** The **number of elements per node** has a **lower bound** and an **upper bound**. These bounds are expressed using the parameter $t \geq 2$, also called the **minimum degree** of the B-tree:

Lower bound: Each node must have **AT LEAST** t **subtrees** as children or equivalently $t - 1$ elements.

- ▶ The only **exception** to this bound is for the **root** node.
- ▶ A root node with **0** elements is an **empty** B-tree.

B-tree properties...continued

- ⑥ All leaves have the **same depth** (from the root).
- ⑦ **IMPORTANT!** The **number of elements per node** has a **lower bound** and an **upper bound**. These bounds are expressed using the parameter $t \geq 2$, also called the **minimum degree** of the B-tree:

Lower bound: Each node must have **AT LEAST** t **subtrees** as children or equivalently $t - 1$ elements.

- ▶ The only **exception** to this bound is for the **root** node.
- ▶ A root node with **0** elements is an **empty** B-tree.
- ▶ root node with ≥ 1 elements is a **non-empty** B-tree.

B-tree properties...continued

- ⑥ All leaves have the **same depth** (from the root).
- ⑦ **IMPORTANT!** The **number of elements per node** has a **lower bound** and an **upper bound**. These bounds are expressed using the parameter $t \geq 2$, also called the **minimum degree** of the B-tree:

Lower bound: Each node must have **AT LEAST** t **subtrees** as children or equivalently $t - 1$ elements.

- ▶ The only **exception** to this bound is for the **root** node.
- ▶ A root node with **0** elements is an **empty** B-tree.
- ▶ root node with ≥ 1 elements is a **non-empty** B-tree.

upper bound: Every node must have **AT MOST** $2t$ subtrees (or equivalently $2t - 1$ elements). A node of a B-tree is **full** if it reaches this bound.

B-tree properties...continued

- ⑥ All leaves have the **same depth** (from the root).
- ⑦ **IMPORTANT!** The **number of elements per node** has a **lower bound** and an **upper bound**. These bounds are expressed using the parameter $t \geq 2$, also called the **minimum degree** of the B-tree:

Lower bound: Each node must have **AT LEAST** t **subtrees** as children or equivalently $t - 1$ elements.

- ▶ The only **exception** to this bound is for the **root** node.
- ▶ A root node with **0** elements is an **empty** B-tree.
- ▶ root node with ≥ 1 elements is a **non-empty** B-tree.

upper bound: Every node must have **AT MOST** $2t$ subtrees (or equivalently $2t - 1$ elements). A node of a B-tree is **full** if it reaches this bound.

B-tree properties...continued

- ⑥ All leaves have the **same depth** (from the root).
- ⑦ **IMPORTANT!** The **number of elements per node** has a **lower bound** and an **upper bound**. These bounds are expressed using the parameter $t \geq 2$, also called the **minimum degree** of the B-tree:

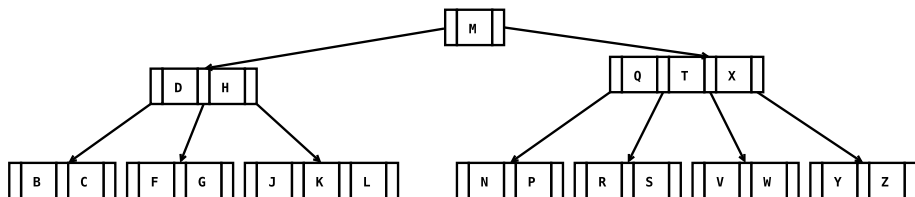
Lower bound: Each node must have **AT LEAST** t **subtrees** as children or equivalently $t - 1$ elements.

- ▶ The only **exception** to this bound is for the **root** node.
- ▶ A root node with **0** elements is an **empty** B-tree.
- ▶ root node with ≥ 1 elements is a **non-empty** B-tree.

upper bound: Every node must have **AT MOST** $2t$ subtrees (or equivalently $2t - 1$ elements). A node of a B-tree is **full** if it reaches this bound.

The simplest B-tree is when $t = 2$. Every internal node has either **2, 3, or 4 subtrees** connecting its **1, 2 or 3 elements**. This is specifically called a **[2-3-4]** tree.

Example B-tree ($t = 3$) containing consonants of the English Alphabet

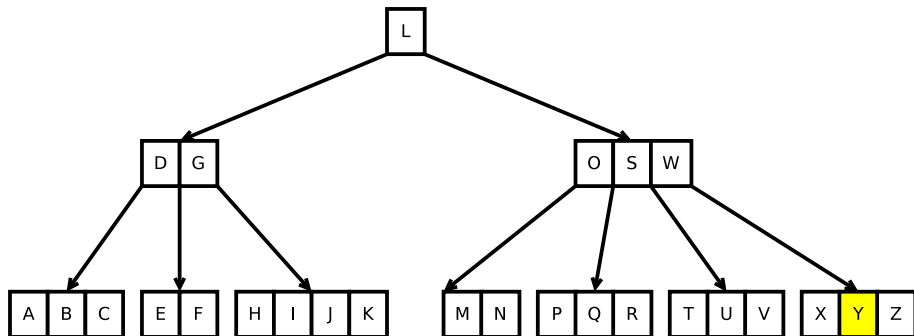


Operation: **Search**(**x**)

Searching for element **x** in a B-tree

- Searching in B-tree is no different from searching in any **binary** search tree.
- However, since each node has $t \geq 2$ children, the decision is no longer **two-way** or **binary**.
- A **multi-way** branching decision has to be made according to its number of children/subtrees.

Search for 'Y' in a B-tree ($t = 3$) made from all letters from the English alphabet.



Operation: **Insert**(**x**)

Inserting into a B-tree

- It requires traversing from the root node to an 'appropriate' **leaf node** to insert into its current list of elements.

Inserting into a B-tree

- It requires traversing from the root node to an 'appropriate' **leaf node** to insert into its current list of elements.
- However, we simply cannot just create a new leaf node with one element, or insert into an existing leaf node that is already full – both **flout** the B-tree properties (see slide 14).

Inserting into a B-tree

- It requires traversing from the root node to an 'appropriate' **leaf node** to insert into its current list of elements.
- However, we simply cannot just create a new leaf node with one element, or insert into an existing leaf node that is already full – both **flout** the B-tree properties (see slide 14).
- In B-trees, a new key is inserted into an **existing leaf node**.

Inserting into a B-tree

- It requires traversing from the root node to an 'appropriate' **leaf node** to insert into its current list of elements.
- However, we simply cannot just create a new leaf node with one element, or insert into an existing leaf node that is already full – both **flout** the B-tree properties (see slide 14).
- In B-trees, a new key is inserted into an **existing leaf node**.
 - ▶ However, this is possible only if the leaf node is **NOT** full.

Inserting into a B-tree

- It requires traversing from the root node to an 'appropriate' **leaf node** to insert into its current list of elements.
- However, we simply cannot just create a new leaf node with one element, or insert into an existing leaf node that is already full – both **flout** the B-tree properties (see slide 14).
- In B-trees, a new key is inserted into an **existing leaf node**.
 - ▶ However, this is possible only if the leaf node is **NOT** full.
 - ▶ If the leaf node is **full**

Inserting into a B-tree

- It requires traversing from the root node to an 'appropriate' **leaf node** to insert into its current list of elements.
- However, we simply cannot just create a new leaf node with one element, or insert into an existing leaf node that is already full – both **flout** the B-tree properties (see slide 14).
- In B-trees, a new key is inserted into an **existing leaf node**.
 - ▶ However, this is possible only if the leaf node is **NOT** full.
 - ▶ If the leaf node is **full**
 - ★ The node is **split** around its **median** forming 2 nodes.

Inserting into a B-tree

- It requires traversing from the root node to an 'appropriate' **leaf node** to insert into its current list of elements.
- However, we simply cannot just create a new leaf node with one element, or insert into an existing leaf node that is already full – both **flout** the B-tree properties (see slide 14).
- In B-trees, a new key is inserted into an **existing leaf node**.
 - ▶ However, this is possible only if the leaf node is **NOT** full.
 - ▶ If the leaf node is **full**
 - ★ The node is **split** around its **median** forming 2 nodes.
 - ★ The median element **moves up** to the **parent node** of the original leaf node, to identify the dividing point of the two split nodes.

Inserting into a B-tree

- It requires traversing from the root node to an 'appropriate' **leaf node** to insert into its current list of elements.
- However, we simply cannot just create a new leaf node with one element, or insert into an existing leaf node that is already full – both **flout** the B-tree properties (see slide 14).
- In B-trees, a new key is inserted into an **existing leaf node**.
 - ▶ However, this is possible only if the leaf node is **NOT** full.
 - ▶ If the leaf node is **full**
 - ★ The node is **split** around its **median** forming 2 nodes.
 - ★ The median element **moves up** to the **parent node** of the original leaf node, to identify the dividing point of the two split nodes.
 - ★ If the parent node were in-turn **full**, this would result in **splitting** the parent now to make room for the incoming median element from leaf (potentially leading to a cascade of splits all the ways up the root).

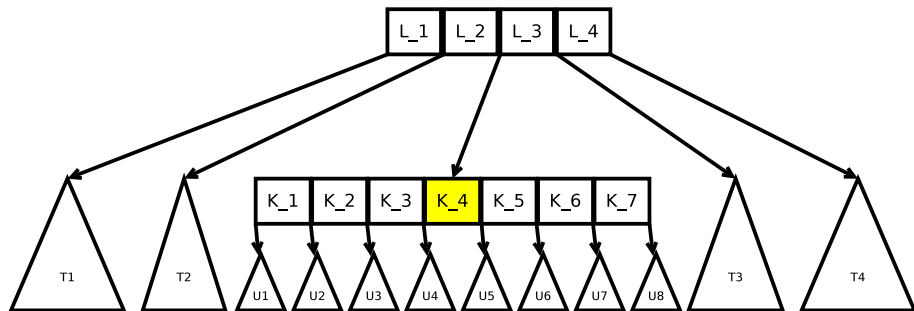
Inserting into a B-tree

- It requires traversing from the root node to an 'appropriate' **leaf node** to insert into its current list of elements.
- However, we simply cannot just create a new leaf node with one element, or insert into an existing leaf node that is already full – both **flout** the B-tree properties (see slide 14).
- In B-trees, a new key is inserted into an **existing leaf node**.
 - ▶ However, this is possible only if the leaf node is **NOT** full.
 - ▶ If the leaf node is **full**
 - ★ The node is **split** around its **median** forming 2 nodes.
 - ★ The median element **moves up** to the **parent node** of the original leaf node, to identify the dividing point of the two split nodes.
 - ★ If the parent node were in-turn **full**, this would result in **splitting** the parent now to make room for the incoming median element from leaf (potentially leading to a cascade of splits all the ways up the root).
 - ★ To make the above logic more efficient in practice, one can proactively **never step on a full node (without first splitting it) during traversal** from the root node to the appropriate leaf node during insert.

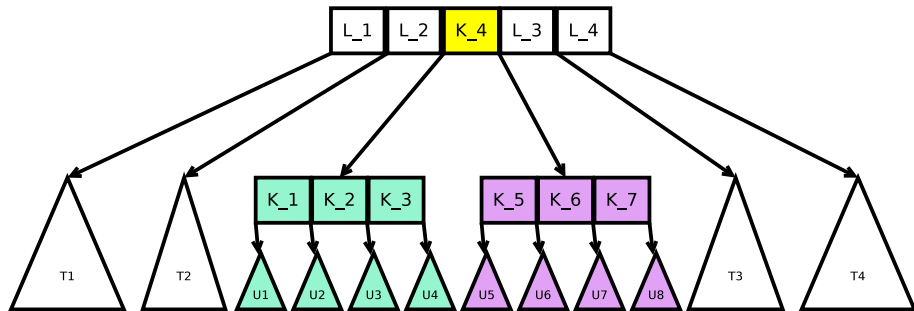
Inserting into a B-tree

- It requires traversing from the root node to an 'appropriate' **leaf node** to insert into its current list of elements.
- However, we simply cannot just create a new leaf node with one element, or insert into an existing leaf node that is already full – both **flout** the B-tree properties (see slide 14).
- In B-trees, a new key is inserted into an **existing leaf node**.
 - ▶ However, this is possible only if the leaf node is **NOT** full.
 - ▶ If the leaf node is **full**
 - ★ The node is **split** around its **median** forming 2 nodes.
 - ★ The median element **moves up** to the **parent node** of the original leaf node, to identify the dividing point of the two split nodes.
 - ★ If the parent node were in-turn **full**, this would result in **splitting** the parent now to make room for the incoming median element from leaf (potentially leading to a cascade of splits all the ways up the root).
 - ★ To make the above logic more efficient in practice, one can proactively **never step on a full node (without first splitting it) during traversal** from the root node to the appropriate leaf node during insert.
 - ★ In other words, when traversing from root to leaf in a B-tree, **split** any **full parent node** of a (sub)tree along the insertion path.

Splitting illustration of any B-tree ($t = 4$) node



Splitting illustration of any B-tree ($t = 4$) node



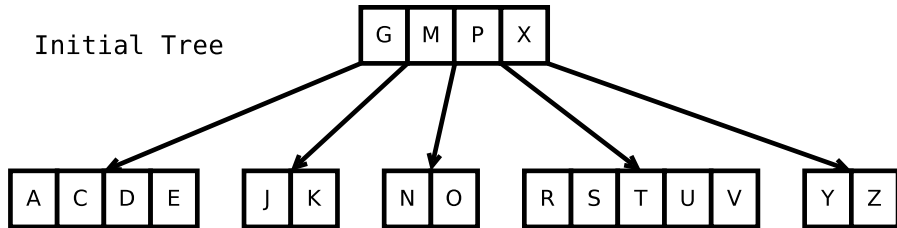
Example: Inserting into a B-tree of degree $t = 3$

Note:

Lower bound on the number of **elements**: $t - 1 = 2$ (except root node)

Upper bound on the number of **elements**: $2t - 1 = 5$

Initial Tree

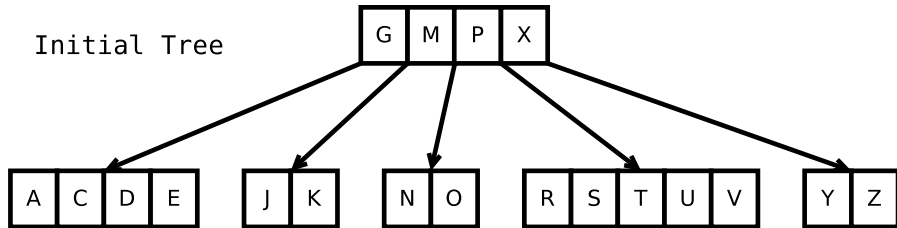


Example: Inserting into a B-tree of degree $t = 3$

Note:

Lower bound on the number of **elements**: $t - 1 = 2$ (except root node)

Upper bound on the number of **elements**: $2t - 1 = 5$

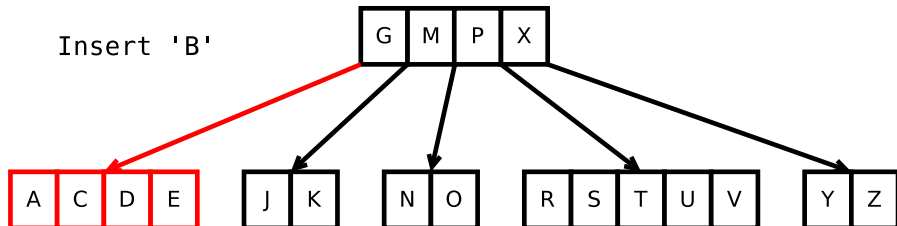


Next

insert('B') into this tree.

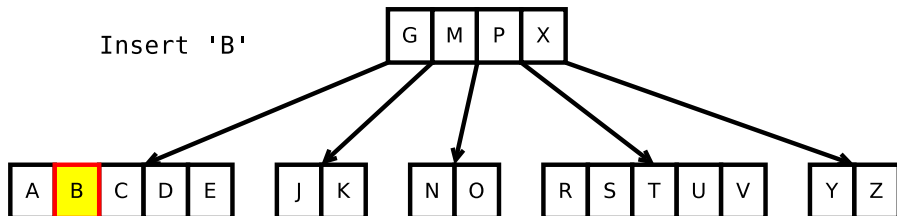
Inserting into a B-tree of degree $t = 3$

To insert element **B**, traverse along an 'appropriate' subtree



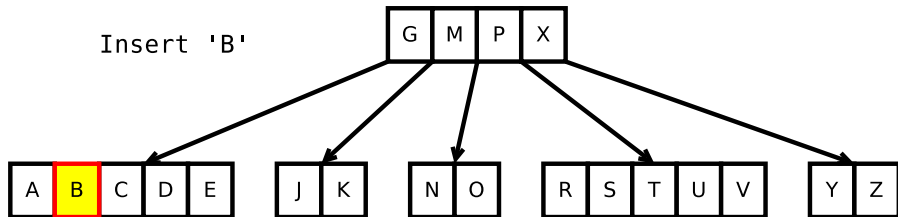
Inserting into a B-tree of degree $t = 3$

Insert 'B'



Inserting into a B-tree of degree $t = 3$

Insert 'B'

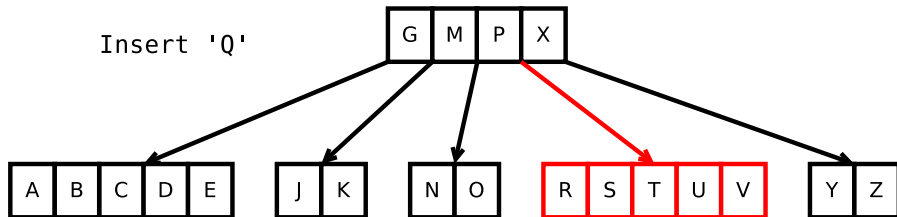


Next

insert('Q') into this tree.

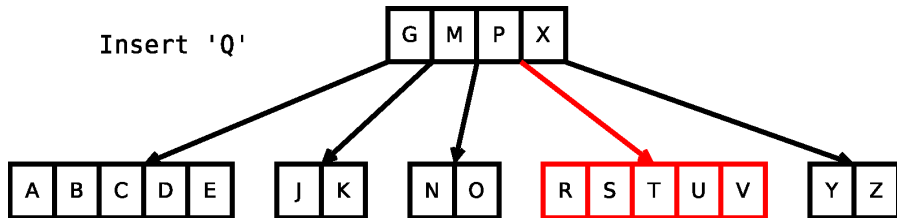
Inserting into a B-tree of degree $t = 3$

Insert 'Q'



Inserting into a B-tree of degree $t = 3$

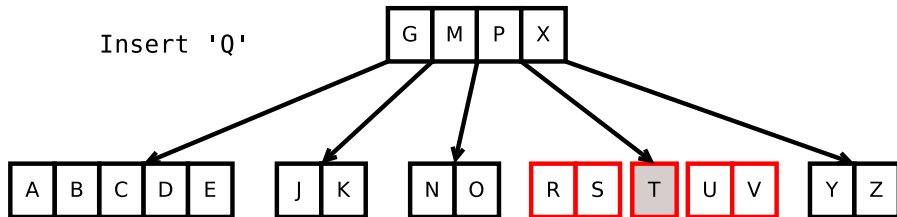
Insert 'Q'



Cannot stop here
as node has max number
of elements in it

Inserting into a B-tree of degree $t = 3$

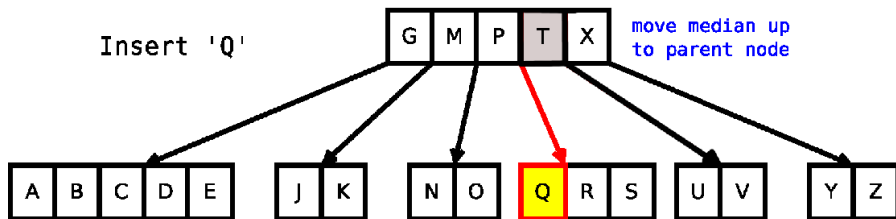
Insert 'Q'



Spilt this node
about the median
element (here 'T')

Inserting into a B-tree of degree $t = 3$

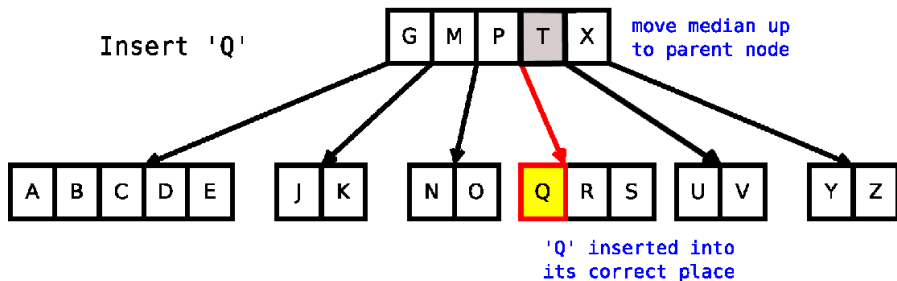
Insert 'Q'



'Q' inserted into
its correct place

Inserting into a B-tree of degree $t = 3$

Insert 'Q'

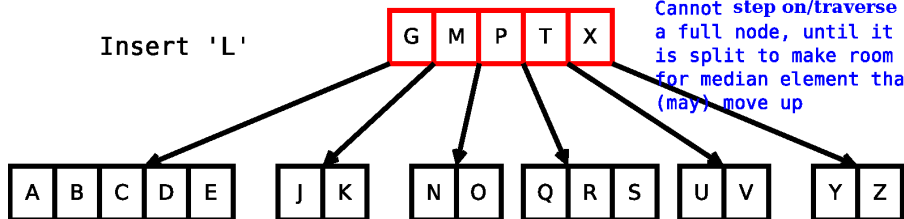


Next

insert('L') into this tree.

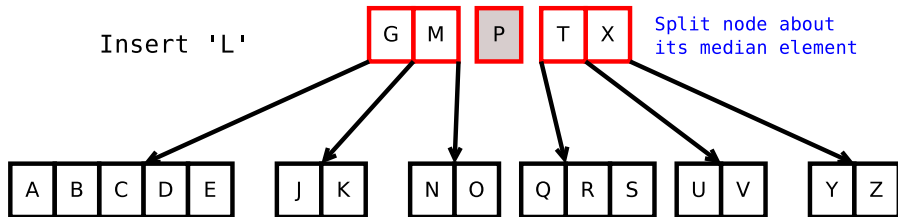
Inserting into a B-tree of degree $t = 3$

Insert 'L'

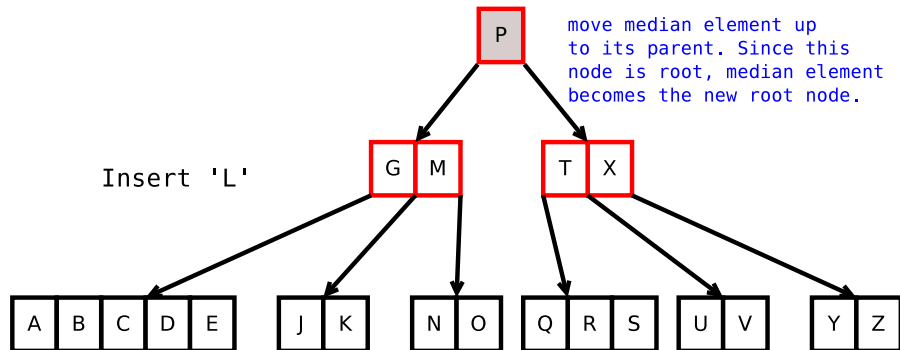


Inserting into a B-tree of degree $t = 3$

Insert 'L'

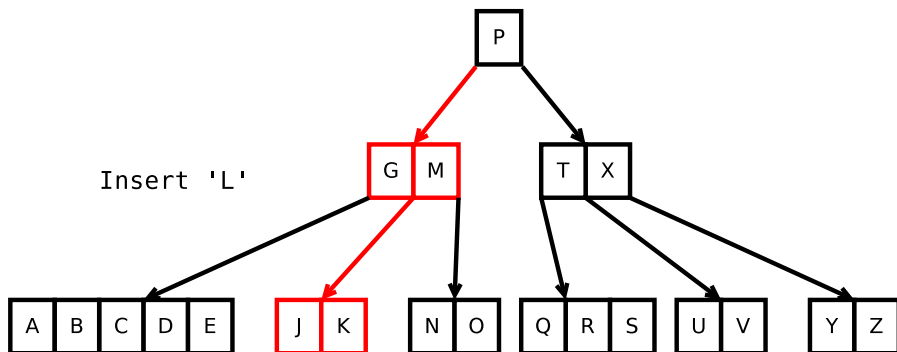


Inserting into a B-tree of degree $t = 3$



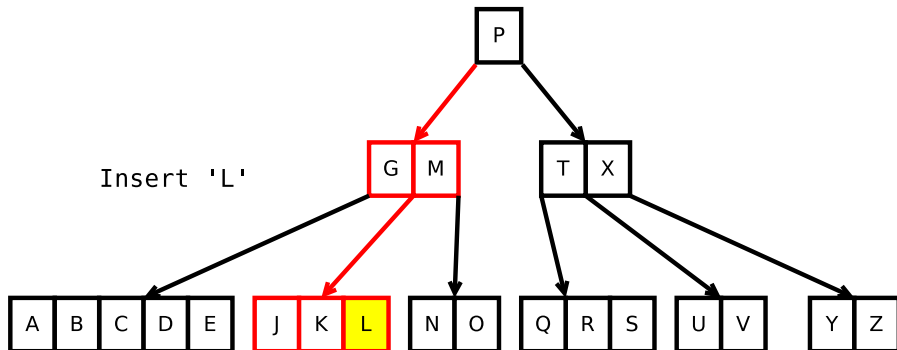
Inserting into a B-tree of degree $t = 3$

Insert 'L'

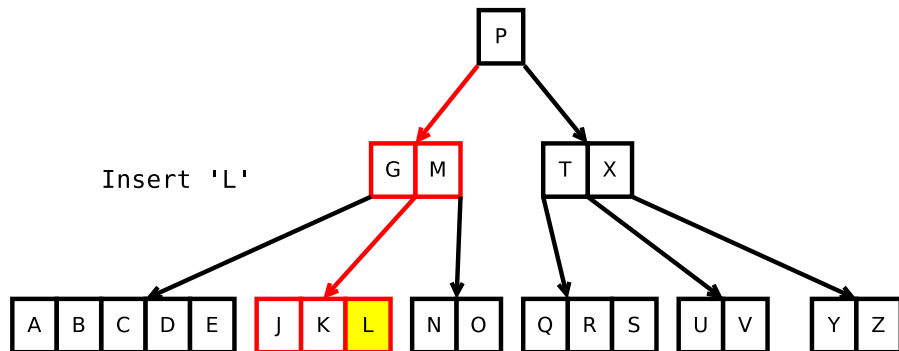


Inserting into a B-tree of degree $t = 3$

Insert 'L'



Inserting into a B-tree of degree $t = 3$

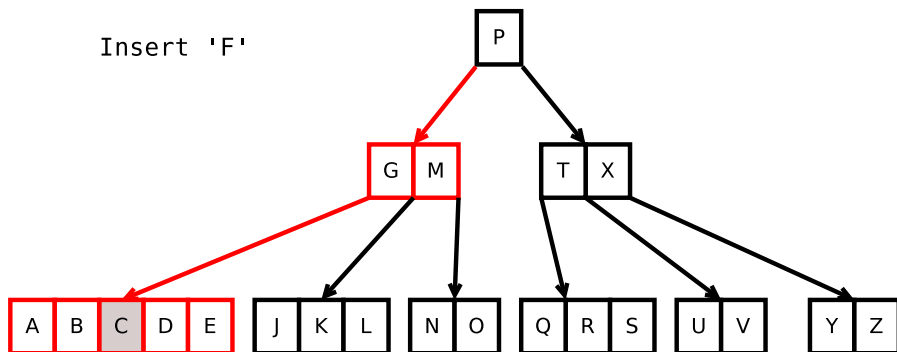


Next

insert('F') into this tree.

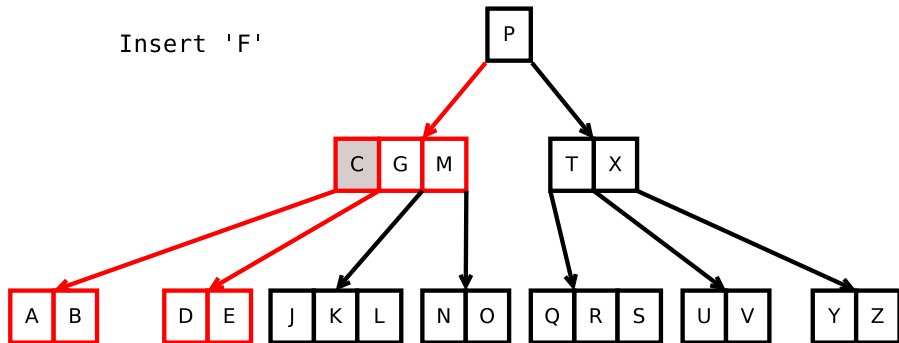
Inserting into a B-tree of degree $t = 3$

Insert 'F'



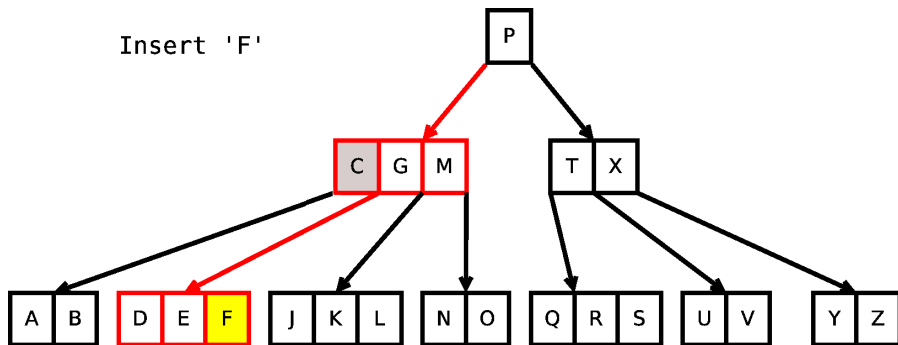
Inserting into a B-tree of degree $t = 3$

Insert 'F'



Inserting into a B-tree of degree $t = 3$

Insert 'F'



Deletion from a B-tree

Remember this during deletion

When traversing down from the root, **never step on any B-tree node** that has a bare-minimum (i.e., $t - 1$) number of elements (details in slides below). Note: Only **root node** is exempted from this rule.

Deleting **x** from a B-tree – Case 1

- **Case 1:** If **x** belongs to a **leaf node** with **strictly more than** minimum number of elements ($> t - 1$) elements. Delete **x** – **trivial!**

*

*All deletes in a B-tree are handled eventually by only this case. Other two deletion cases (shown in later slides) convert their deletion scenarios into an eventual Case 1 delete problem.

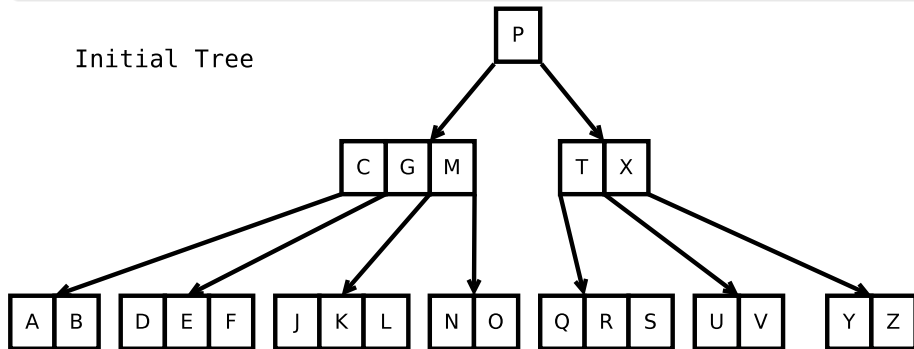
Deleting from a B-Tree (degree $t = 3$): Running examples

Note:

Number of elements **lowerbound** $t - 1 = 2$ (except the root node)

Number of elements **upperbound** $2t - 1 = 5$

Initial Tree



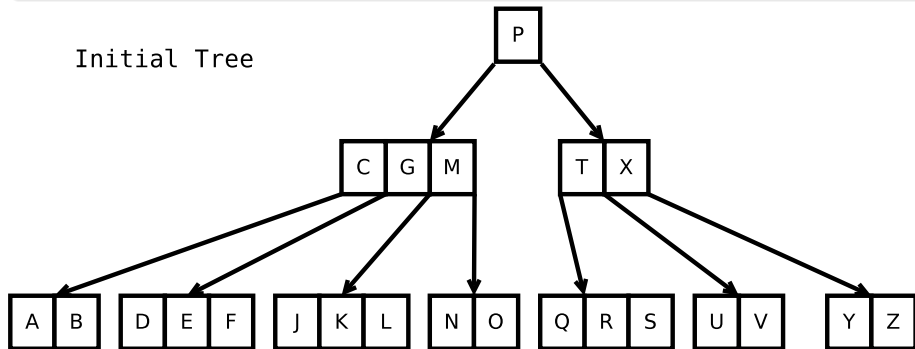
Deleting from a B-Tree (degree $t = 3$): Running examples

Note:

Number of elements **lowerbound** $t - 1 = 2$ (except the root node)

Number of elements **upperbound** $2t - 1 = 5$

Initial Tree



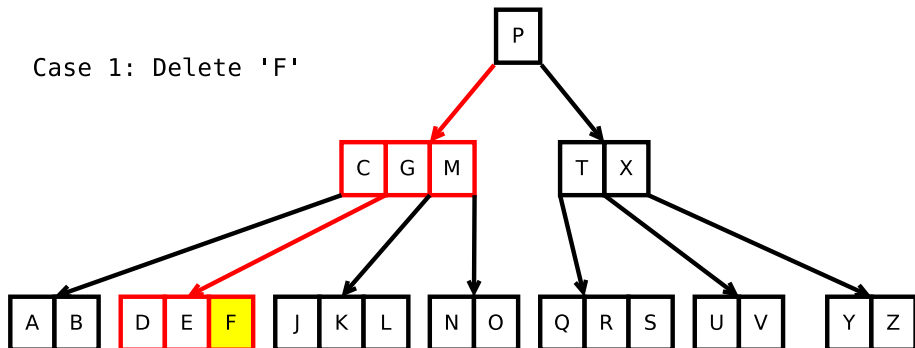
Next

delete('F') from this tree.

Delete 'F' – Case 1 example

- 1 $x='F'$ belongs to a **leaf node** with $> t - 1 = 2$ elements.

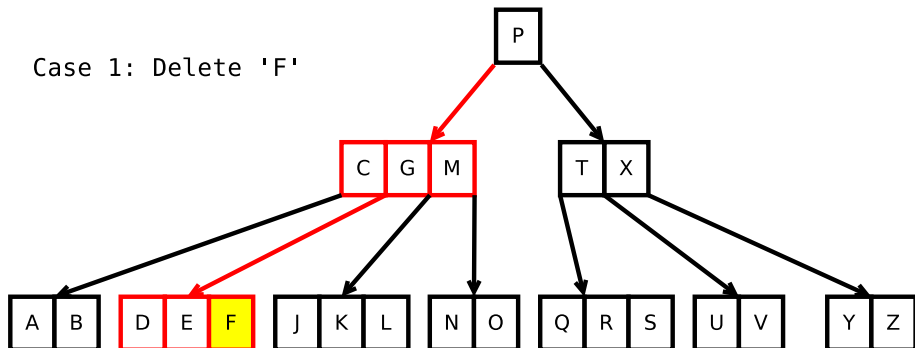
Case 1: Delete 'F'



Delete 'F' – Case 1 example

- ❶ $x = 'F'$ belongs to a **leaf node** with $> t - 1 = 2$ elements.

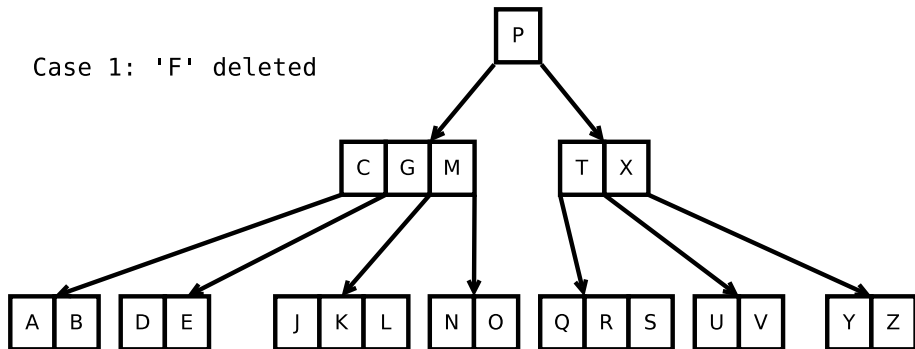
Case 1: Delete 'F'



Since 'F' is a leaf node with $> t - 1 (= 2)$ elements, deleting 'F' involves just removing that element – **trivial!**

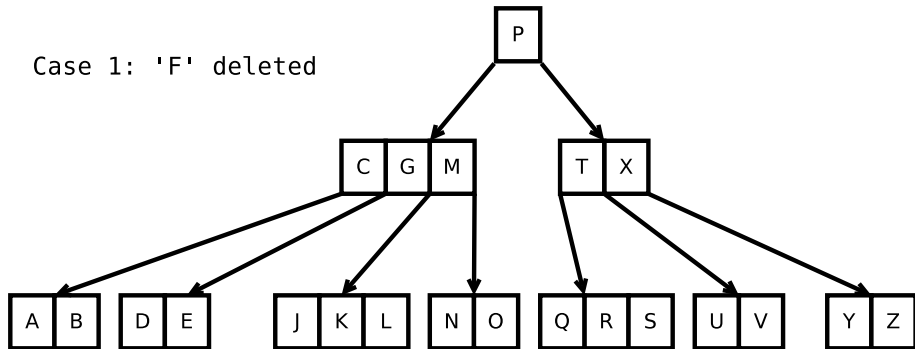
Delete 'F' – Case 1 example

Case 1: 'F' deleted



Delete 'F' – Case 1 example

Case 1: 'F' deleted



Next

delete('M') from this tree.

Deleting **x** from a B-tree – Case 2

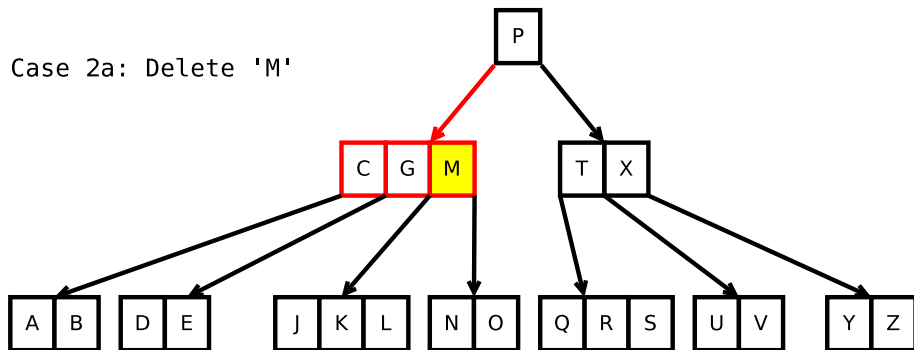
- **Case 2:** If **x** belongs to an **internal node**:
 - Ⓐ If the left child node of **x** has at least t elements, then find the **in-order predecessor** (say, **w**) in the left subtree, replace **x** with **w**, and then recursively delete **w** in the left subtree.
 - Ⓑ Else if the right child node of **x** has at least t elements, then find the **in-order successor** (say, **y**) in the right subtree, replace **x** with **y**, and then recursively delete **y** in the right subtree.
 - Ⓒ Else, both **left** and **right** child nodes of **x** have **exactly** $t - 1$ elements, then **merge** **x** and child nodes, and recursively delete **x**.

Delete 'M' – Case 2a example

2 $x = 'M'$ belongs to an **internal node**:

- (a) If the left child node of x has at least t elements, then find the **in-order predecessor** (say, $w = 'L'$ in this example) in the left subtree, replace x with w , and then recursively delete w in the left subtree.

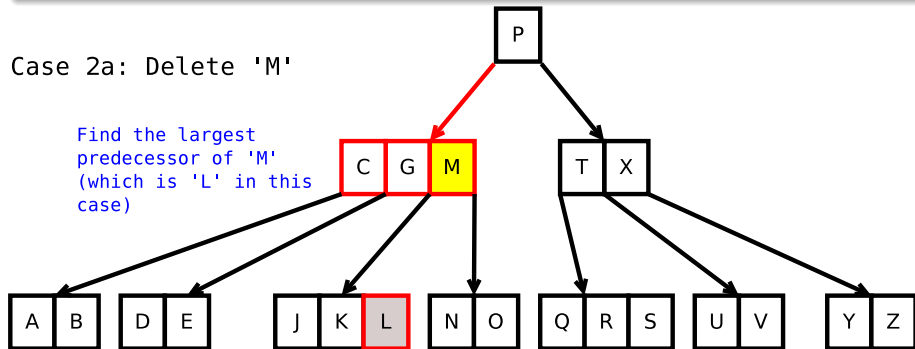
Case 2a: Delete 'M'



Delete 'M' – Case 2a example (continued)

In-order predecessor of any element **x** in the B-tree is the **largest** element that is $\leq x$. This can be derived by traversing the left subtree of **x** and accessing the **rightmost** element in that subtree.

Case 2a: Delete 'M'

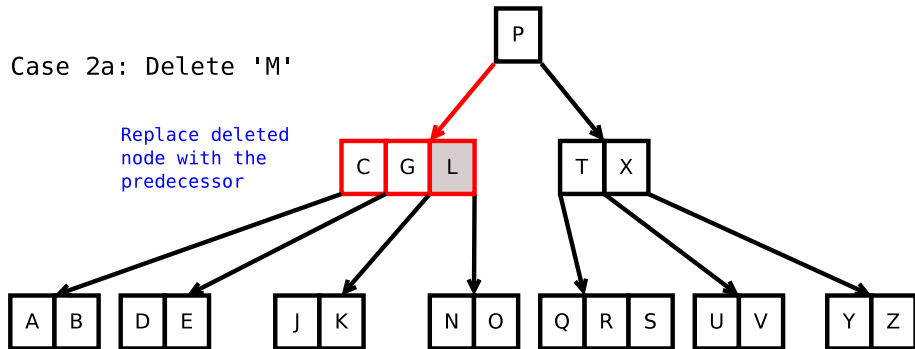


In this example, the **in-order predecessor** of 'M' is 'L'. To delete 'M', replace 'M' with its **predecessor** 'L', and (recursively) delete 'L' in the left subtree (next slide)

Delete 'M' – Case 2a example (continued)

Case 2a: Delete 'M'

Replace deleted
node with the
predecessor



Case 2b mirrors Case 2a

2 If x belongs to an **internal node**:

- (b) ... if the right child node of x has at least t elements, then find the **in-order successor** (say, y) in the right subtree, replace x with y , and then recursively delete y in the right subtree.

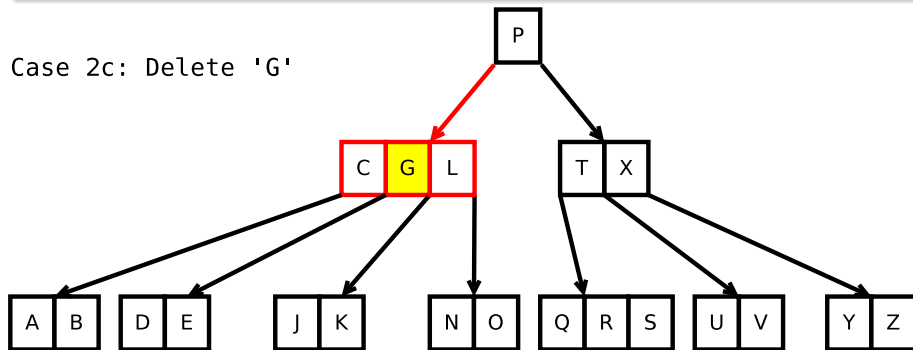
In-order successor of any element x in the B-tree is the **smallest** element that is $> x$. This can be derived by traversing the right subtree of x and accessing the **leftmost** element in that subtree.

No explicit example provided to demonstrate Case 2b, as it is simply a symmetric (i.e. a mirror operation) to Case 2a, whose example we have seen in the previous slide.

Delete 'G' – Case 2c example

- 2 If $x = 'G'$ belongs to an **internal node**:
- Ⓒ and both **left** and **right** child nodes of x have **exactly** $t - 1 = 2$ elements, then **merge** x and child nodes, and recursively delete x .

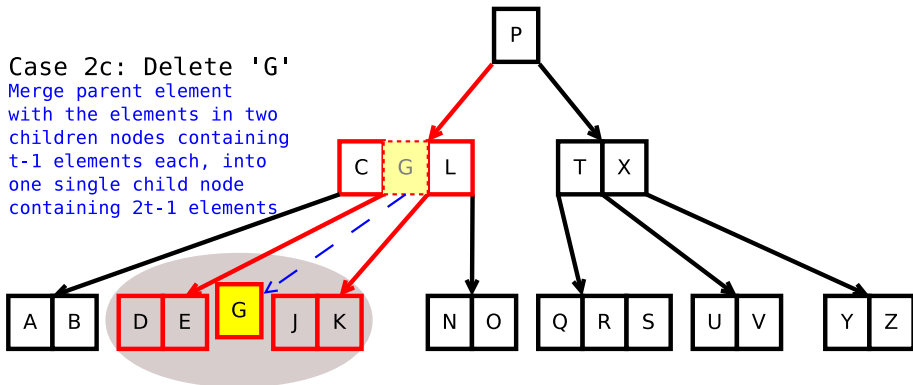
Case 2c: Delete 'G'



Delete 'G' – Case 2c example (continued)

Case 2c: Delete 'G'

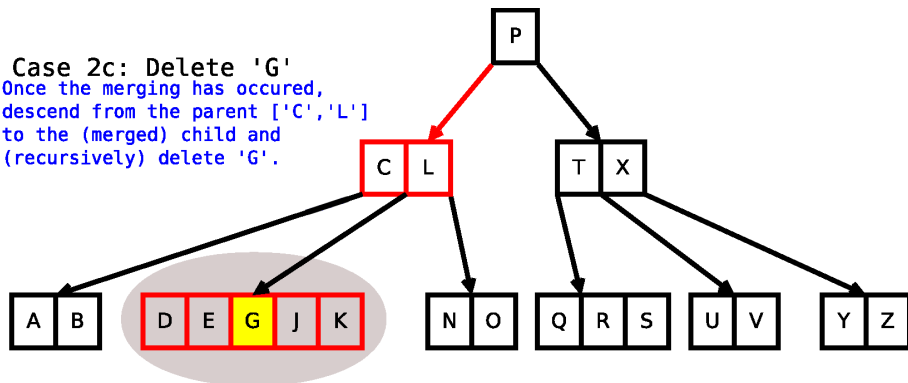
Merge parent element
with the elements in two
children nodes containing
 $t-1$ elements each, into
one single child node
containing $2t-1$ elements



Delete 'G' – Case 2c example (continued)

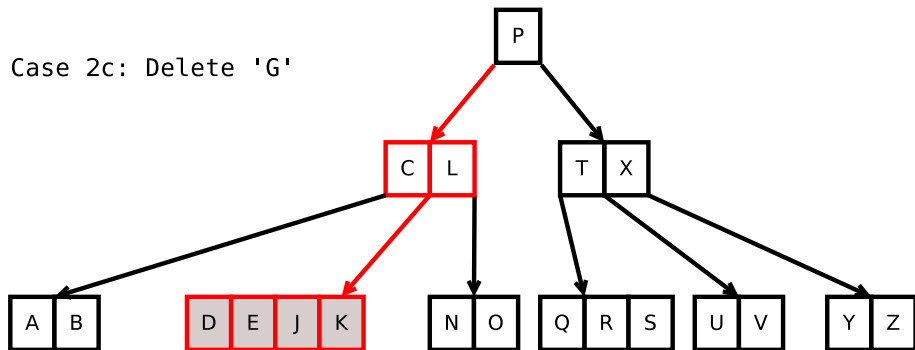
Case 2c: Delete 'G'

Once the merging has occurred,
descend from the parent ['C','L']
to the (merged) child and
(recursively) delete 'G'.



Delete 'G' – Case 2c example (continued)

Case 2c: Delete 'G'



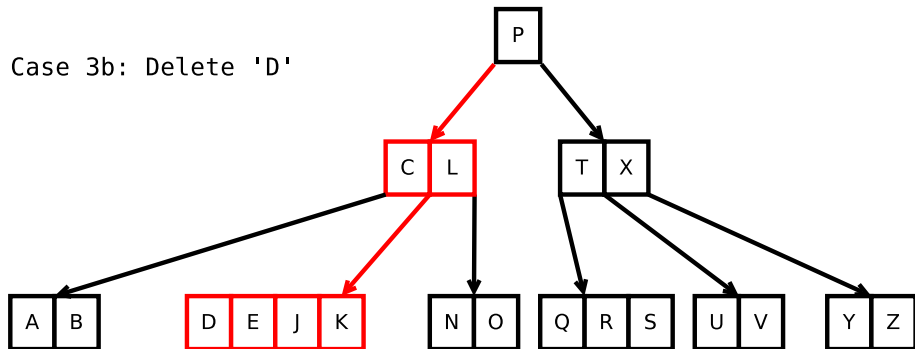
Deleting **x** from a B-tree – Case 3

This situation arises when, during traversal from the root to locate the node to be deleted, the traversal is stalled because it encounters a node that contains the minimum allowable number of elements (i.e., $t - 1$). This case specifically addresses how to handle such scenarios to ensure that deletion can proceed correctly without violating the tree's properties.

- **Case 3:** If the **traversal** is stopped because the 'appropriate' subtree containing **x** has a node with **exactly** $t - 1$ elements, then :
 - Ⓐ if this subtree's '**immediate sibling**' has **at least** t elements, give an extra element to the appropriate subtree by **rotating** the predecessor or successor element **within the sibling node** to parent, followed by moving the original parent element to the appropriate subtree.
(refer to the example on slide 58)
 - Ⓑ else, if its **immediate sibling** also has **exactly** $t - 1$ elements, merge the parent element with both sibling elements to form a single node.
This **may** reduce the height of the tree.
(refer to the example on slide 53)

Delete 'D' – Case 3b example[†]

Case 3b: Delete 'D'

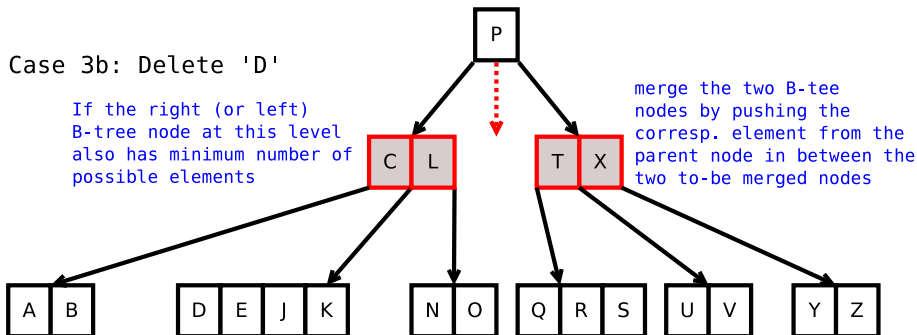


[†]we will first see case 3b before case 3a, stay tuned!

Delete 'D' – Case 3b example[†] (continued)

Case 3b: Delete 'D'

If the right (or left)
B-tree node at this level
also has minimum number of
possible elements



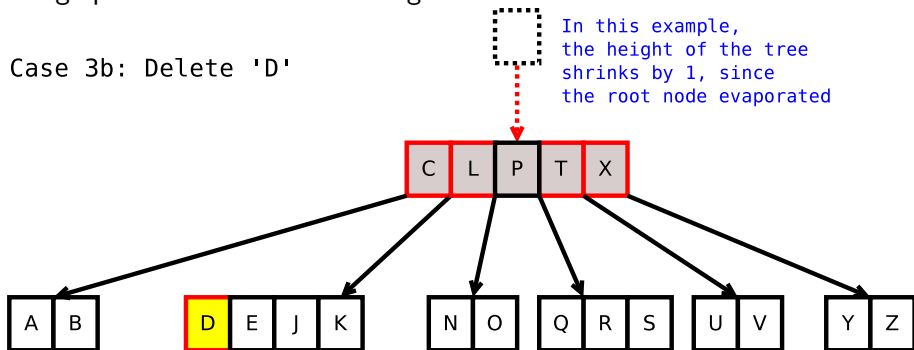
merge the two B-tree
nodes by pushing the
corresp. element from the
parent node in between the
two to-be merged nodes

[†]we will first see case 3b before case 3a, stay tuned!

Delete 'D' – Case 3b example[†] (continued)

Merge parent element with siblings.

Case 3b: Delete 'D'

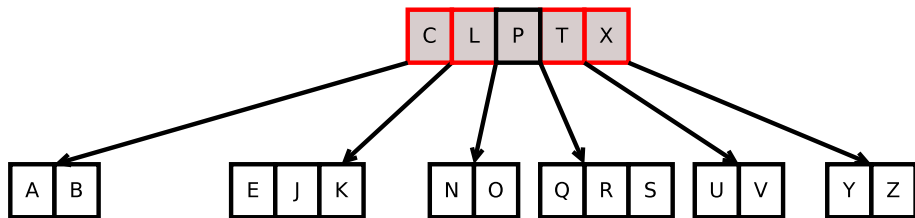


[†]we will first see case 3b before case 3a, stay tuned!

Delete 'D' – Case 3b example[†] (continued)

Recursively delete 'D'

Case 3b: Delete 'D'

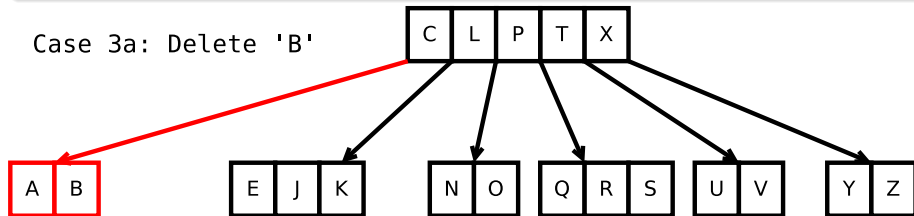


[†]we will first see case 3b before case 3a, stay tuned!

Delete 'B' – Case 3a example

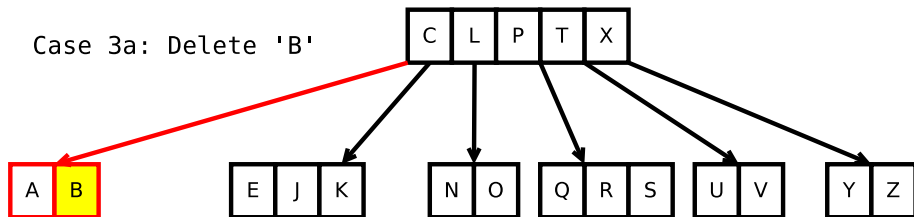
- 3 If the **traversal** is stopped because the 'appropriate' subtree containing **x** has a node with **exactly** $t - 1$ elements, then:
 - a if this subtree's '**immediate sibling**' has **at least** t elements, give an extra element to the appropriate subtree by **rotating** the predecessor or successor **within the sibling node** to its parent, followed by moving the original parent element to the appropriate subtree.

Case 3a: Delete 'B'



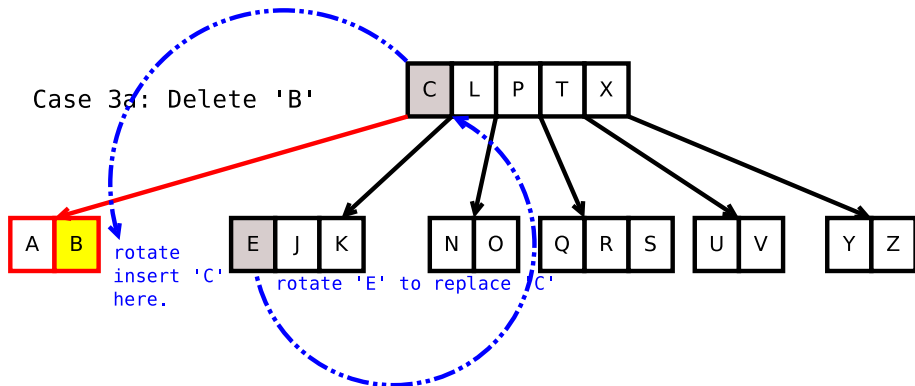
Delete 'B' – Case 3a example (continued)

Case 3a: Delete 'B'



Cannot descend into a minimal node with exactly $t-1$ elements in it. In this case, 'Immediate sibling' node (to the right ['E', 'J', 'K']) has at least t elements in it. Rotate the immediate successor ('E') of the parent element 'C' into the parent node, followed by pushing the parent element 'C' into the appropriate ['A', 'B'] subtree, so that it now has at least t elements. By doing this, the algorithm can descend into this node/subtree.

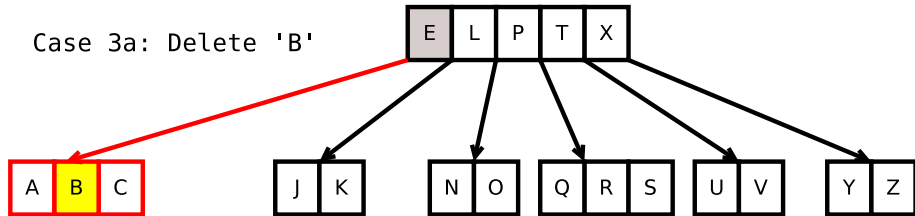
Delete 'B' – Case 3a example (continued)



[†]Question: How you would handle the case when [E, J, K] (and other currently leaf-level nodes) were all internal nodes of a B-tree and had their own children? Specifically, what would happen to the left-tree (before rotation) of 'E' in such a scenario? Hint: Take inspiration from the rotations you learnt for AVL trees in FIT2004!

Delete 'B' – Case 3a example (continued)

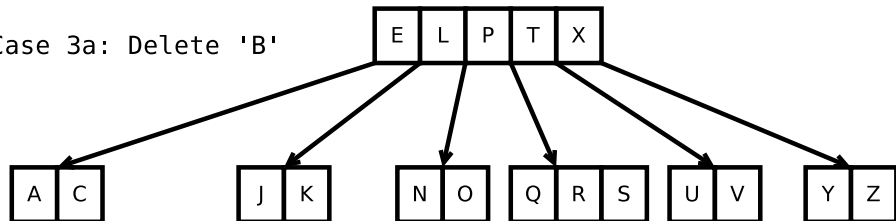
Case 3a: Delete 'B'



Now descend (recursively)
into the appropriate
subtree and delete 'B'

Delete 'B' – Case 3a example (continued)

Case 3a: Delete 'B'



B- tree – Space and Time complexities

Height bounds of any B-tree with parameter t

Lecture will show that for any B-tree with parameter t (meaning each node (except the root) has a minimum degree of t and a maximum degree of $2t$) its height h has both upper and lower bounds of $O(\log_t N)$, where N is the number of elements in the tree.

This bound ensures the following worst-cases.

	Average case	Worst case
--	--------------	------------

Time Complexity of operations

Search	$O(\log(n))$	$O(\log(n))$
Insert	$O(\log(n))$	$O(\log(n))$
Delete	$O(\log(n))$	$O(\log(n))$

Space Complexity

	$O(n)$	$O(n)$
--	--------	--------

B-Tree Summary

- B-trees are generalizations of balanced search trees
- Designed to optimize access to secondary storage (eg. hard disks).
- All leaf nodes are at the same height, so insertion and deletion $O(\log(n))$ -time, all cases

coming up...

Week 10 : Linear Programming

--o0o--

END

--o0o--